

Evaluating Virtual Memory Algorithms in the UR-OS Simulator

Gabriela Aldana

School of Sciences and Engineering
Universidad del Rosario
Bogota D.C., Colombia
gabriela.aldana@urosario.edu.co

Valeria Arce

School of Sciences and Engineering
Universidad del Rosario
Bogota D.C., Colombia
valeria.arce@urosario.edu.co

Santiago Jorigua

School of Sciences and Engineering
Universidad del Rosario
Bogota D.C., Colombia
santiago.jorigua@urosario.edu.co

Samuel Flores

School of Sciences and Engineering
Universidad del Rosario
Bogota D.C., Colombia
samuel.flores@urosario.edu.co

Pedro Wightman

School of Sciences and Engineering
Universidad del Rosario
Bogota D.C., Colombia
pedro.wightman@urosario.edu.co

Abstract—The UR-OS operating system simulator is an academic tool designed to support the teaching and learning of concepts in the area of Operating Systems. This simulator incorporates a complex structure that emulates active elements of a computer system (processor, main memory, swap memory, interrupts) and of an operating system (process management, memory addressing, etc.). In its current version, UR-OS allows modeling virtual memory management algorithms, like First-in-First-out (FIFO), Least Recently Used (LRU), among others. The main characteristic of this simulator is that it is a very simple tool to work with and modify, and it is freely available for academic activities and research. In this work, two new virtual memory algorithms are introduced: Second-chance Least-Frequently-Used (SCLFU) and Least Recently/Frequently Used (LRFU). These techniques were tested over three distinct scenarios: sequential, random, and patterned memory accesses. Results show that mainly in random scenarios, both LFU and LRFU reduce the page faults in the testing scenario, compared to the rest of the algorithms.

Index Terms—Simulation, Operating Systems, Memory Management, Virtual Memory

I. INTRODUCTION

Virtual memory management is a very critical subsystem of all modern operating systems. Understanding its functionality and impact on the performance of process execution is very important for any computer science student.

Most commercial operating systems allow the parameterization of certain aspects of the virtual memory algorithms, but the implementation is already done, and it is not easy to observe. In an academic environment, it is very important to expose students to scenarios where their abstractions and assumptions can be put to the test.

Within this context, the UR-OS simulator [1], [2] emerges as a valuable pedagogical tool. Building upon its initial success in facilitating the understanding of process management and main memory management, the simulator has been significantly enhanced to encompass the critical domain of Virtual Memory Management.

UR-OS 0.0.6 incorporates a comprehensive suite of functionalities designed to provide students with hands-on experience in navigating the complexities of virtual memory management, as well as improvements in process modeling and other internal features. This simulator can also be used for early prototyping of new algorithms, which can be later used to test new approaches on actual operating systems.

This work presents an initial experience to evaluate and compare new virtual memory algorithms in the UR-OS simulator. The paper introduces two algorithms developed by students of the Operating Systems class of the Applied Mathematics and Computer Science bachelor's program at Universidad del Rosario, in Bogota, Colombia, during the first semester of 2025. The algorithms are compared in terms of the total number of page faults under three scenarios: sequential, random, and patterned memory access which try to model average process behavior.

II. RELATED WORK

Creating and using operating systems simulation tools to help in the classroom has been explored in the past; however, their use is not as common as it should be.

In the literature, there is a long list of simulation platforms. The SOSimulador [3] introduces a graphical interface for process scheduling and memory management.

The simulator in [4] presents a web-based simulator that covers the most relevant subsystems of an OS. It has several libraries and allows students to upload their codes for testing and sharing. The website MOSS [5] offers a collection of topic-specific simulators for all operating systems sub-systems.

In [6], the authors present the Basoper simulator and the evaluation of the impact on the students. The simulator allows the creation of test scenarios for process scheduling on a web-based interface, including individual cycle types to create bursts. In [7] the authors present the SIME simulation

platform, specifically designed for teaching memory management, including a graphical interface that illustrates how the algorithms work and a theoretical explanation of the topics.

Existing simulation tools for teaching Operating Systems vary in their approaches: some of them prioritize visual representation to illustrate concepts and just support understanding, while others aim for a deeper emulation of the computer system's internal structure, allowing the students to experience the implementation of the algorithm and facing the associated challenges. The usefulness of either approach depends on the desired level of involvement of the students in the process of experimentation.

UR-OS was designed to give students a functional platform that integrates the main elements of a computer system, in which they can implement the algorithms and see how the components relate to each other. The general structure of the simulator was designed to accompany the lectures of the classic textbook by Silberschatz [8], both for process planning, memory allocation, and free memory management algorithms, as well as virtual memory. Its current structure is presented in the Fig. 1.

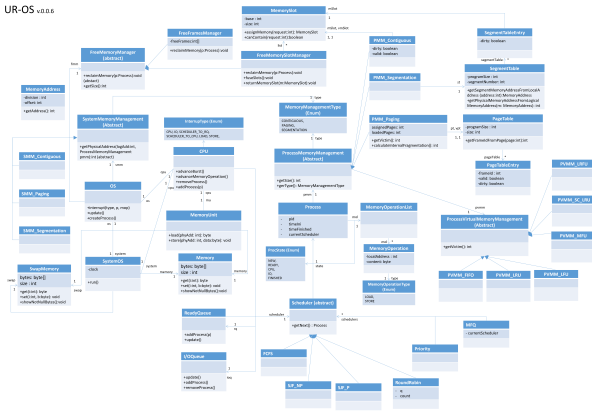


Fig. 1. Structure of the UR-OS

III. VIRTUAL MEMORY MANAGEMENT ALGORITHMS IN UR-OS

The UR-OS simulator models the implementation of some classic virtual memory algorithms for victim page selection; this is, when a process has a limited amount of pages assigned in memory and they are all full, but a new page is referenced, then one of the pages in memory needs to go back to the swap area on the hard drive.

The simulator includes an implementation of First-In-First-Out (FIFO) and requires students to implement the Least-Recently-Used (LRU), Least-Frequently-Used (LFU), and Most-Frequently-Used (MFU).

This simulator allows the verification of the system's behavior and provides insight into its performance under key operating conditions.

The variables that were used in this work are:

- 1) **Page Replacement Algorithm** The system allows the selection of different algorithms to test, under the same conditions, their performance. Their behavior was analyzed specifically in victim selection upon page faults.
- 2) **Frames per Process** Each process was constrained to a fixed number of frames, between 2 and 4. This metric evaluates how well the system handles memory allocation under limited frame availability.
- 3) **Process Load Time (t_{init}):** Processes were dynamically created at specific clock times. The system's ability to manage CPU and memory usage under concurrent process loads was assessed.
- 4) **Instruction Types and Volume** Each process executed a defined sequence of instructions, including:
 - CPU instructions (processor usage).
 - MEMORY instructions (e.g., LOAD, STORE) accessing virtual addresses.

This helped evaluate different behaviors in terms of memory access: random memory access, fully sequential and patterned access to data structures distributed along the program's logical address space, and their impact on system performance.

In addition, the simulator allows the measurement of performance indicators like:

- 1) **Page Faults** The simulation measured how many memory accesses triggered page faults and how effectively the replacement algorithm responded to the access pattern.
- 2) **Execution Time:** The total execution time, measured in clock cycles from t_{init} to $t_{finished}$, was recorded to evaluate system efficiency.
- 3) **Memory Accesses and Loaded Pages** Each memory access is logged along with the corresponding page. The simulation distinguished between valid and invalid accesses, and tracked data reads and writes.
- 4) **Execution Queues (CPU and IO)** The CPU queue behavior was monitored to identify periods of processor activity and idle time. Although IO instructions are implemented, they were not utilized in this simulation.
- 5) **Page States** For each page, the simulation monitored:
 - Whether the page was currently loaded in memory (Valid bit).
 - Whether it had been modified (Dirty bit).
 - Its assigned memory frame.

This is an example of how to structure a page replacement algorithm, using the FIFO strategy.

Algorithm 1 FIFO-Based Page Replacement Simulation

Initialize simulation:

clock $\leftarrow$ 0

Load process list from input file

Initialize memory, page table, FIFO queue

Start simulation loop:

while there are active or pending processes **do**

Check if new processes arrive $\rightarrow$ add to ready queue

if CPU is free **then**

Assign next process from ready queue to CPU

end if

Execute next instruction of current process

if instruction requires memory access **then**

if page not in memory **then**

Trigger page fault

if memory full **then**

Remove page at front of FIFO queue

end if

Load required page and add to FIFO queue

end if

end if

if process completes **then**

Release memory and CPU

end if

Advance clock by 1 unit

end while

Output simulation results and performance summary

Students were asked to propose and implement new algorithms and compare them in terms of page faults, under different conditions of memory accesses. The authors implemented two new approaches: Least-Recently-Frequently-Used (LRFU) and Second-Chance-LFU (SCLFU). Each of these introduces different management mechanisms that directly impact the system's efficiency and accuracy when selecting which page to replace during a page fault.

A. SCLFU (Second-Chance Least Frequently Used)

This algorithm is based on the classic Least Frequently Used (LFU), where the system keeps count of the times a page has been accessed and, when a victim must be selected, the operating system will look for the page with the lowest count.

To improve replacement decision, a variant of the LFU algorithm was developed that incorporates a **second chance mechanism**. This algorithm, called SCLFU, combines two criteria: usage frequency and a Boolean bit indicating whether a page has been accessed recently.

The process begins by identifying the most recently loaded pages in memory, considering only the latest ones. Then, the complete access history is scanned to count how many times each of these pages has been used, and those with recent accesses are marked. In the final stage, the algorithm selects as the victim the page with the lowest frequency. However, if this page has its second chance bit active, it is given an additional round and reinserted at the end of the list. Only

if it is selected again without the active bit will it finally be replaced.

Algorithm 2 SCLFU - Second Chance LFU

Initialize frequency, secondChance, loadedPages

1. Get last loaded unique pages:

Traverse memoryAccesses from the end

Add unique pages to loadedPages until full

2. Compute frequency and second chance:

for each page in memoryAccesses **do**

if page in loadedPages **then**

Increment frequency[page]

Set secondChance[page] = true

end if

end for

3. Victim selection:

while true **do**

Find page with lowest frequency

if secondChance is active **then**

Reset secondChance bit

Reinsert page at end of loadedPages

else

Return page as victim

end if

end while

This second algorithm introduces a more complex criterion: it not only considers how many times a page has been accessed but also whether it has been used recently. Thus, SCLFU aims to avoid hasty replacement decisions by incorporating what could be called "intelligent tolerance."

- In the first phase, the system scans the recent access history and identifies the currently loaded pages, limiting itself to the most recent ones.
- Then, it calculates how many times each of these pages has been accessed and assigns a "second chance" bit if the page has been used recently.
- Finally, the algorithm selects as a candidate the page with the lowest usage frequency. If it has its second chance bit active, it is given another round (the bit is reset and the page is reinserted at the end of the set). Only if it is again the least used without the second chance bit will it be replaced.

B. LRFU (Least Recently/Frequently Used)

The LRFU algorithm combines the strengths of recency-based and frequency-based replacement strategies. It assigns a dynamic score to each page using a decaying function that accounts for both how often and how recently a page has been accessed. The algorithm uses a decay factor $\lambda \in (0, 1)$ to exponentially reduce the weight of older accesses, which ensures that frequent but outdated accesses contribute less than recent ones. This balance makes LRFU highly adaptable, particularly in scenarios with shifting memory access patterns.

Unlike LRU, which may evict frequently used pages if they were not recently accessed, or LFU, which may retain stale but

historically frequent pages, LRFU dynamically adjusts each page's importance based on a decaying access history. The page with the lowest LRFU score is selected as the victim during replacement.

Algorithm 3 LRFU - Least Recently/Frequently Used

getVictim(memoryAccesses, loaded):

```
Initialize lrfuValues map
For each page in memory:
    Traverse access history and update score:
        score = score *  $\lambda^{\Delta t}$  + 1
Select the page with the lowest score as the victim
```

LRFU requires maintaining access histories or timestamps and applying exponential decay calculations, which increases its computational complexity compared to FIFO or LRU. However, this extra cost results in a more intelligent eviction policy, especially beneficial in dynamic or mixed workloads.

- When a page is accessed, its score is incremented, with prior scores decaying based on how long ago they occurred.
- A decay factor λ controls the influence of past accesses. Lower λ gives more weight to recency; higher λ favors long-term frequency.
- During page replacement, the page with the lowest decayed score (least recent and least frequent) is selected as the victim.

Despite its higher complexity, LRFU provides superior adaptability and efficiency in memory management, particularly under workloads where access patterns evolve over time.

IV. COMPARISON OF VIRTUAL MEMORY MANAGEMENT IMPLEMENTATIONS

The comparison of the algorithms, in terms of their performance, required a certain level of automation to make the process efficient. Based on this premise, some adaptations were included in the simulator.

A. Simulation Automation Module

An automation component was designed to run multiple tests without requiring manual intervention. This functionality was developed within the `BatchMemoryComparison` module, whose main purpose is to orchestrate the parallel execution of different page replacement policies under a common base scenario.

The procedure begins with the definition of a shared configuration for all experiments. This configuration includes essential parameters such as page size, total number of pages, process scheduling policy (e.g., First Come First Served), and the input file containing the description of the processes to be simulated. In this way, the only variable that changes between executions is the memory algorithm used, ensuring a fair and controlled comparison.

The most relevant element of the adopted approach is the concurrent execution of experiments. For each algorithm, a

new execution thread is created, responsible for initializing the system with the corresponding configuration and running the complete simulation process. This modular and parallelizable structure not only improves efficiency in terms of execution time, but also facilitates scalability of the environment, allowing future expansion of the algorithm set or scenarios without altering the core logic.

Algorithm 4 Simulation Automation of Page Replacement Algorithms

1. Define base configuration:

```
pageSize ← 256
numPages ← 8
schedulingPolicy ← "FCFS"
processFile ← "inputs/procesos1.txt"
Create baseConfig with above
```

2. Define list of algorithms:

```
memoryAlgorithms ← [FIFO, LRU, LFU, LRFU, SCLFU, MRU]
```

3. For each algorithm:

```
for currentAlgorithm in memoryAlgorithms do
```

```
    Create thread:
```

```
        Print separator
```

```
        Print "Starting simulation with currentAlgorithm"
```

```
        Instantiate SystemOS(baseConfig, currentAlgorithm, null)
```

```
        Run .run()
```

```
        Start thread
```

```
end for
```

1) *Simulation Scenarios*: The simulator used to have some hard-wired, simple manual testing scenarios; however, to be able to see changes in behavior in page replacement algorithms, a more efficient way to create complex scenarios was needed.

A robust simulation generator was developed using Python, capable of handling longer programs, introducing the possibility to emulate iterative memory access behavior. This tool was designed to evaluate the performance of the virtual memory algorithms. Three different experiments were conducted: one with sequential accesses to memory, one page after the other; one with random accesses; and one with patterned access to data structures along the process.

2) *Sequential Access Scenario* (`Robust.py`): This scenario was designed to represent a highly structured and memory-intensive workload. Each generated process systematically traverses its entire assigned virtual memory region, performing read (LOAD) and write (STORE) instructions in regular 512-byte steps.

Each process accesses six consecutive pages (192 KB), first executing a full sweep with sequential instructions and then a second pass with mixed instructions interleaved with simulated CPU cycles. The processes are launched in a staggered manner (every 3 time units), allowing the evaluation of concurrent system behavior.

Experimental Purpose: To assess the performance of the FIFO algorithm under a high-intensity sequential load, observing the page fault frequency during the initial phases and its progressive stabilization as pages are reused.

3) *Random access Scenario (Fallos Aleatorios.py)*: This scenario simulates memory access patterns similar to real-world, unpredictable workloads. Unlike the previous case, each process performs 500 memory accesses randomly selected within ranges that simulate matrices (A, B, C, D). Each access is associated with a randomly selected LOAD or STORE instruction and interleaved with CPU operations to simulate processing between accesses.

Additionally, the processes perform accesses to specific addresses representing temporary variables, generating extra traffic and promoting page faults in scattered memory regions.

Experimental Purpose: To evaluate the system's ability to handle chaotic memory access patterns, with emphasis on the latency caused by frequent page faults and the effectiveness of the FIFO algorithm under unpredictable workloads.

4) *Patterned access Scenario (Fallos Fijos.py)*: This third scenario emulates a computational load typical of scientific or matrix-processing algorithms, where memory access patterns are highly regular and predictable. Each process accesses multiple memory regions (simulating matrices A to F) in an orderly manner, performing initialization, cumulative reading, and intermediate writing to temporary variables.

Accesses are structured into nested loops over indices i and j , reproducing a controlled sequential behavior over disjoint memory regions.

Experimental Purpose: To evaluate the FIFO algorithm's response to structured and deterministic memory access patterns. This scenario allows analysis of the system's ability to keep the most frequently used pages in memory when access follows a predictable path.

To analyze how page replacement algorithms behave under different memory conditions, three comparative graphs were generated. Each graph represents a distinct experimental scenario: sequential (Fig. 2), patterned (Fig. 3), and random (Fig. 4) access behaviors. They all illustrate the relationship between the number of allocated memory frames and the number of page faults.

The sequential scenario can be assumed as a control scenario, in which the number of page faults should be constant among all algorithms, given that there will be no repetition on any page reference, making the replacement decision irrelevant. In this case, all algorithms indeed have the same number of page faults for each number of assigned frames. In addition, it can be seen how the number of total page faults decreases with more assigned frames, as expected.

The patterned behavior has a somewhat similar behavior to the one shown in the sequential one, but it starts reusing pages in a very well-defined pattern. Some small differences can be seen among the algorithms, showing a slight advantage of the FIFO and the MFU, which both benefit the new pages by taking out the older ones or the ones that have been used

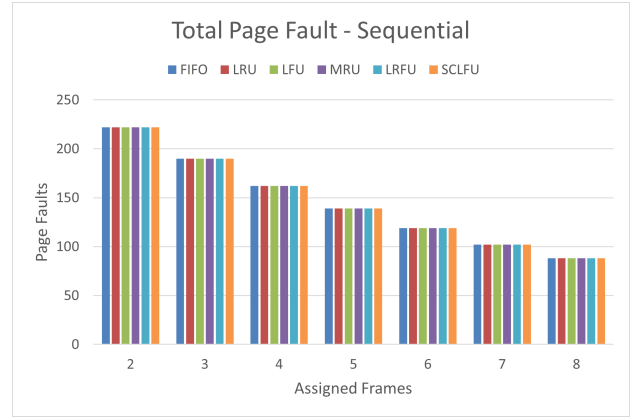


Fig. 2. Total page faults – Sequential access pattern

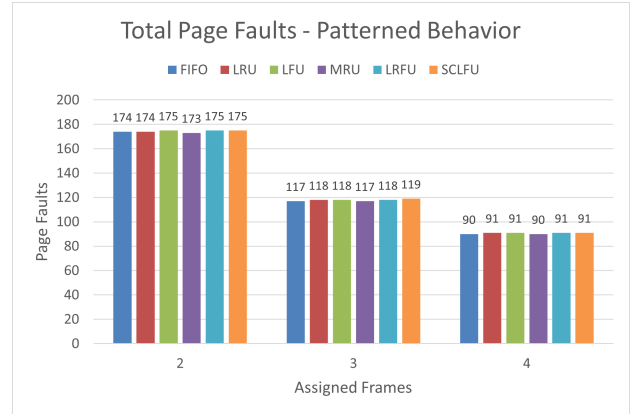


Fig. 3. Total page faults – Patterned access pattern

more extensively (and that perhaps have been in the system for a while).

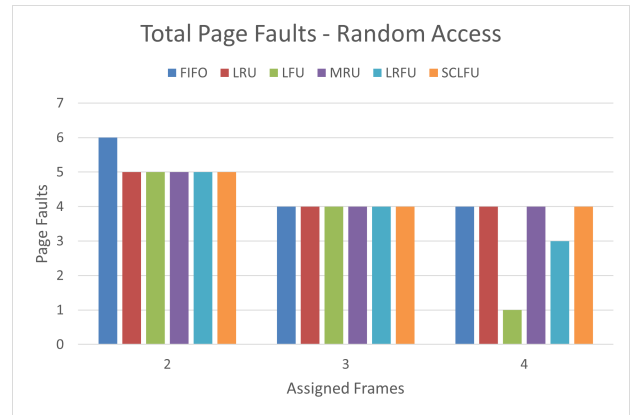


Fig. 4. Total page faults – Random access pattern

Finally, in the random access results, it can be seen that some algorithms indeed show great improvement due to the decisions they make. The ones that present the best performance are LFU and LRFU when processes have 4 assigned frames. These algorithms discard pages that have not been

accessed for a while, so it benefits the ones that perhaps are still being used.

V. CONCLUSIONS AND FUTURE WORK

The UR-OS simulator presents a feasible tool to support the learning experience of operating systems in computer science programs. This work presented the results of a hands-on learning experience in which students proposed and tested new algorithms under valid scenarios.

The paper presented tests over six page replacement algorithms, including classic ones like FIFO, LRU, LFU, MFU, and two new ones SCLFU, and LFRU, created by the students. In addition, a more automated approach for generating realistic scenarios and automating the testing process. The automated simulation module enabled parallel testing with consistent configurations, ensuring fair and reproducible comparisons.

The results show that, especially for random memory access scenarios, the LFU and the LRFU algorithm had the best results, due to their preference to replace pages that, most likely, are no longer useful for the execution.

Overall, this work provides a foundation for the design, evaluation, and iterative enhancement of educational operating systems, with a strong emphasis on hands-on experimentation and a deeper understanding of memory management.

The future work, which is more of an ongoing commitment with the platform, the students, and the community worldwide, is to continue the implementation of new functionalities and improve the infrastructure. Also, hybrid algorithms, scalability under heavier workloads, and integration with real-time scheduling policies could be considered for a future version.

CODE AVAILABILITY

The UR-OS platform is a Java-based Netbeans project. The public source code for students is available in the following Github repository: https://github.com/pedrowightman/ur_os_public_vm. This version implements some algorithms to serve as examples for students, which can be required to implement the rest of the algorithms and new ones. In addition, the implementation of the performance indicators for process planning is not included. All faculty and researchers interested in accessing the code with all implementations can request access via email and must give credit for authorship in future related work.

REFERENCES

- [1] P. Wightman, "Teaching process management with an operating systems simulator: The ur_os experience," in *Proceedings of the IEEE Biennial Congress of Argentina - ARGENCON*. New York, NY, USA: IEEE, 2024, p. 287–292.
- [2] —, "Enhancing the ur-os simulator for teaching memory management," in *2024 IEEE 42nd Central America and Panama Convention (CONCA-PAN XLII)*, 2024, pp. 1–6.
- [3] A. Costa, C. Morais, A. Freitas, J. Belmiro, R. Rocha, I. Vanderlei, K. Camboim, and J. Araujo, "Using visual resources to assist in the teaching-learning process of operating systems through the development of a simulator," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.
- [4] H. Pham, "A web-based interactive and visualized approach to simulations of operating systems," in *2022 5th International Conference on Information and Computer Technologies (ICICT)*, 2022, pp. 148–152.
- [5] R. Ontko and A. Reeder, "Modern operating systems simulators," <https://www.ontko.com/moss/>, accessed: 2010-09-30.
- [6] H. Y. Shishido, L. J. Azevedo, P. S. L. d. Souza, J. C. Estrella, and S. M. Bruschi, "Basoper: a web-based open educational resource for the teaching of scheduling algorithms for batch systems," in *Congreso Internacional de Informática Educativa*, 2015, pp. 498–503.
- [7] R. Lopes, D. A. de Souza, J. R. B. de Carvalho, W. O. Silva, and V. L. P. de Sousa, "Sime: Memory simulator for the teaching of operating systems," in *2012 International Symposium on Computers in Education (SIIE)*, 2012, pp. 1–5.
- [8] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. John Wiley Sons, Inc., 2018.